Understanding reinforcement learning for G1 escort control

ECE 598JK Humanoid Escort Research | 6 November 2026 | Beginner guide

The current G1 escort controller succeeds because imitation supplied a useful neural policy and conservative PPO updates preserved it. This guide builds the reinforcement learning concepts from the beginning, explains how the techniques relate, and separates what the experiments show from what remains unproven.

What we are teaching the robot

Our task is to accompany a moving client, normally 1.5 metres behind them. When the client turns, the robot should turn with them. When the client stops, the robot should settle into the correct formation. Simply walking, reaching the endpoint, or collecting a large reward is not enough.

There are two learned controllers, with very different jobs. The high-level escort actor chooses forward speed, sideways speed and turning rate. The low-level NVIDIA AGILE gait turns those requests into joint targets that make the full Unitree G1 walk and balance in MuJoCo.

Escort actor at 5 Hz

Sees the current escort situation and requests body-frame motion.

Frozen AGILE at 50 Hz

Uses robot state and history to generate walking joint targets.

MuJoCo at 500 Hz

Integrates forces, joints, balance and actual ground contacts.

Think of learning to drive a car whose engine controller already works. We are learning how to follow someone safely along a route, rather than relearning how combustion works. Here the separation is locomotion versus escort decisions. Freezing the gait lets us study the high-level problem while keeping the walking mechanism fixed.

The high-level actor never receives authority to set motor torques or teleport the robot. Its commands pass through the existing gait, PD control and limits. Requested speed is therefore not necessarily achieved speed: measured sustained forward speed was about 0.407 m/s near the 0.6 m/s software command cap. Stage 1 clients use conservative 0.15–0.25 m/s peak-speed settings.

Three ways to obtain a controller

A controller is a rule for deciding what to do next. We can write that rule, fit it to examples, or improve it from trial outcomes. These approaches can be combined.

Approach	What teaches it	What changes
Handwritten control	A person specifies an equation, such as moving toward a desired offset.	No neural learning is required.
Imitation learning	Examples of the action a teacher would choose for an observation.	A neural actor learns to reproduce the teacher.
Reinforcement learning	Rewards and outcomes from the actions the actor actually tries.	The actor learns which actions produce better future returns.

Imagine a driving instructor. Imitation is copying the instructor's steering decisions. Reinforcement learning is trying decisions and receiving feedback about progress, positioning and failures. The instructor can supply a useful starting point; experience can then refine it.

Our successful method is **imitation-pretrained PPO**. A diagnostic follower first supplied action labels. A neural actor learned those labels. PPO then fine-tuned that actor using reward. This is still a reinforcement-learning fine-tuning experiment, but it is not evidence that PPO discovered escorting from scratch.

The teacher was a simple diagnostic controller using the same causal observation as PPO. It was not a person controlling a robot, a future-route oracle, or the final Reactive Baseline A. Calling it a diagnostic teacher keeps the experimental claim accurate.

The vocabulary of a reinforcement learning episode

An **agent** is the decision maker: our escort actor. The **environment** is everything it interacts with: the G1 physics, scripted client, scene, clock and task evaluator. The **policy** is the actor's mapping from observations to actions.

The true **state** includes much more than the actor sees: every joint, contact, controller buffer and scenario trajectory. An **observation** is the limited numerical snapshot we allow the actor to use. These are not interchangeable. Hiding future route information is intentional: a deployed robot would not know a person's precomputed future path.

One **transition** is a single decision and its consequences: observation, action, reward, next observation and episode-ending flags. Here it lasts 0.2 simulated seconds. An **episode** runs from a deterministic reset through warmup and active escorting to success, failure or timeout. Warmup does not advance the client route or earn task reward.

A **rollout** is a collected sequence of transitions. A **training update** uses those collected numbers to change neural weights. Taking 5,000 environment transitions is not the same as taking 5,000 neural optimizer steps.

Termination means a task outcome such as success, a fall or sustained client loss. **Truncation** means an external cutoff such as the configured time limit. This distinction affects how the critic estimates unfinished future return. A deliberately shortened visualization preview is neither a successful episode nor a new training result.

How this relates to traditional reinforcement learning

A textbook introduction may use a small maze with a table of values for each square and action. That is tabular reinforcement learning. Our robot has continuous positions, velocities and commands, so a table with one row for every possible situation is impractical. Neural networks approximate the decision and value functions instead. The basic feedback loop is still the same: observe, act, receive feedback and repeat.

Traditional action-value methods such as Q-learning estimate how good each action is in a state. Our actor-critic method splits the work: the actor directly proposes a continuous command and the critic estimates the current observation's expected return. These are different implementation choices within reinforcement learning, not different definitions of learning from reward.

PPO is model-free in this experiment. It learns from sampled outcomes rather than learning its own predictive dynamics model and planning through that model. MuJoCo is still a physics model used by the environment. Having a simulator does not make the learning algorithm model-based, and we do not backpropagate derivatives through the simulator or frozen AGILE network.

The hierarchy is engineered: escort commands sit above a fixed walking skill. We are not learning the decomposition, selecting among many learned options, or jointly retraining both levels. Imitation is a supervised initialization step; subsequent PPO is online interaction with the simulator. It is not offline reinforcement learning merely because demonstration data was used earlier.

What the actor sees and what an action means

The actor and critic each receive the same fixed 40-number vector. Numbers are scaled using fixed engineering units and clipped into [-1,1]. No running observation normalizer is added. Changing those scales after training would change the meaning of the learned weights.

Observation group	Numbers	Intuitive meaning
Client position and relative velocity	4	Where the client is and how their motion differs from the robot, in yaw-aligned robot axes.
Client heading and desired offset	4	Sine/cosine of heading difference plus the client-frame target offset.
Validity and freshness flags	2	Whether the client proxy is usable; these are initially tied to simulator validity.
Robot pelvis motion	3	Measured pelvis-origin body-axis forward/lateral velocity and angular z.
Torso gravity and angular velocity	6	Balance orientation and how the torso rotates.
Previous applied action	3	The last executed normalized command.
Three empty pedestrian slots	18	Reserved fixed slots; all zeros and invalid in Stage 1.

Relative client positions use planar robot-heading axes; measured robot twist uses the full pelvis orientation and the pelvis origin. These conventions are explicitly implemented rather than treating every velocity as interchangeable. Body angular z is also not exactly Euler yaw rate when the robot tilts; the stopping evaluator uses the documented yaw-rate measurement.

An action is three normalized values. Multiplying by (0.6,0.6,0.9) produces requested (vx,vy,yaw rate) in m/s, m/s and rad/s. For example, (0.5,0,0) requests 0.3 m/s forward. These are body-frame commands: forward rotates with the robot rather than pointing permanently along world x.

The actor sees current and past numerical information, never route completion, future waypoints, scenario family labels or evaluator metadata. The renderer may show a human, but the policy is still observing a simulator-pose proxy, not images or a vision tracker.

How reward describes the task

A **reward** is feedback for one transition. A **return** adds rewards over time, with distant rewards usually discounted. A good reward makes desirable long-term behavior attractive; it does not directly tell the network which action to take.

Objective C is versioned as `tracking_stop_v3`. Let the desired point be 1.5 m behind the client along the client's heading. The formation error is the robot's displacement from that point, expressed in client axes. A positive longitudinal error means the robot is too far forward relative to its preferred position, not necessarily ahead of the client itself.

Component	Current definition	Why it exists
Formation	−dt × robust_cost(error / 0.75 m), capped at 10 before multiplying by dt.	Prefer the target offset throughout the episode.
Velocity tracking	−dt × robust_cost(relative target-point velocity error / 0.25 m/s), capped at 4.	Track motion and slow with the desired formation point.
Progress	+4 × newly credited accompanied route distance in metres.	Make genuine escort progress instead of waiting forever.
Command change	−0.02 × squared change in applied normalized action.	Discourage unnecessary abrupt command changes.
Torso stability	−0.02 × dt × nonnegative torso tilt measure.	Small balance-related shaping.
Terminal outcome	+20 for success; −100 for a task-ending failure.	Value successful completion and penalize failures.

The robust cost is z^2 when z is at most one and $2z-1$ above one, subject to its cap. Close to the target it gives a smooth quadratic preference; far away it grows less sharply than an uncapped square. These are experimental task scales, not universal optimal settings.

The velocity term uses the robot's movement and desired-point movement over the just-finished interval. The target point can move when the client turns even if client centre speed is similar. Computing this inside the evaluator does not expose future paths to the actor.

Progress uses two nondecreasing ledgers: the furthest raw robot progress and the furthest progress bounded by the current desired following point. Credit requires both to advance and requires valid accompaniment. Eligibility also checks error at most 2 m, client distance at most 3 m and not being ahead of the client. Racing through an ineligible location consumes its raw progress; returning later cannot farm it again. A stationary robot cannot earn progress merely because the client advances.

For an illustrative 0.2 s transition, formation error 0.30 m contributes −0.032; velocity error 0.05 m/s contributes −0.008; 0.04 m of eligible new progress contributes +0.16. A single-axis normalized command change of 0.1 costs −0.0002. With no other terms the arithmetic gives +0.1198. This example explains scales; it is not a measured episode.

A high return is not proof of escort success. We separately evaluate formation, stopping, failures and completion. Earlier reward investigations found it necessary to test tempting shortcuts, rather than assume a large reward always means the intended behavior.

Success means accompanying and then settling

The client has to reach the route endpoint, and the G1 has to complete meaningful robot and accompanied progress: at least 60% of the route length under the configured checks. The robot must restore formation within 0.75 m and complete the stopping hold within the five-second recovery window.

The one-second hold allows ordinary gait micromotion. It requires net drift over the whole window at most 0.05 m/s, maximum sampled excursion at most 0.10 m, native planar speed at most 0.15 m/s, and absolute Euler yaw rate at most 0.1 rad/s, while formation/progress conditions remain qualified. This is a sampled test at the environment update rate, not a continuous physical guarantee.

Client distance above 3 m continuously for three seconds terminates as client loss. Falls, supervisor failures, boundary violations and irrecoverable physics failures have explicit reason codes. The active episode time limit is 100 s. A stationary robot cannot simply wait at spawn until the client arrives and claim success.

Notice the relationship: reward supplies frequent learning feedback; the success definition supplies the actual task criterion. Keeping both visible exposes cases where the policy gains some reward yet fails the task.

Actor and critic learn different things

The **actor** answers “what command should I choose?” The **critic** answers “from what I can currently observe, how much discounted return should I expect?” The critic is a learned estimate, not a supervisor with knowledge of the true future.

Both networks use two hidden layers of 128 units with Tanh activation. A hidden layer repeatedly combines the input numbers to represent useful relationships. “MLP” means multilayer perceptron: a feedforward network rather than a memory-based recurrent network. The actor architecture has enough representational capacity to imitate successful Stage 1 behavior; that does not independently prove critic capacity or easy learning from scratch.

The actor outputs a Gaussian distribution over each action coordinate: a mean command and learned standard deviation. During training PPO samples around the mean. During deterministic evaluation it uses the mean. A model can therefore pass deterministic evaluation while broad training noise repeatedly causes failures.

The critic makes learning less noisy by providing a baseline. If an outcome is better than expected, the chosen action receives a positive **advantage**; if worse, a negative one. A rough one-step comparison is reward + discounted next value − current value. Generalized advantage estimation, or GAE, blends such comparisons across multiple time intervals.

We use gamma=0.99 and GAE lambda=0.95. At five decisions per second, gamma=0.99 means a reward 20 seconds away receives a multiplier of about 0.366. This illustrates why delayed terminal penalties can be hard to associate with an early mistake. Dense formation and motion feedback help, but the critic still has to infer outcomes from limited causal observations.

What PPO does and what its safeguards do not do

PPO alternates two activities. First it collects experience using its current stochastic policy. Then it updates actor and critic on minibatches of that experience. It must not change the policy too aggressively while reusing data collected by the old policy.

The probability ratio compares how likely a recorded action is under the new policy versus the policy that collected it. PPO’s clipped objective reduces the incentive for certain overly large ratio changes. Its clip range is 0.2 here. That is not a hard rule saying network parameters, velocities or action means can change by only 20%.

KL divergence measures the change between probability distributions. It is a different concept from clipping. The target-KL option stops further optimization in a block after detecting excessive sampled policy change. A minibatch update can already have occurred before that detection; it is not a mathematical guarantee that every change stays below the target.

Setting	Current guarded value	Meaning
Learning rate	0.00001	Size scale for optimizer parameter updates.
Target KL	0.01	Early-stop guard against excessive distribution movement.
PPO clip range	0.2	Clipped policy-improvement objective.
Rollout length	1024 per worker	Collect this many steps before a full ordinary update.
Workers	2 independent instances	2048 aggregate transitions per ordinary rollout.
Batch size and epochs	256 and up to 10	Minibatch size and maximum passes over collected data.
Entropy coefficient	0.001	Small incentive to retain exploration.
Value coefficient and gradient norm	0.5 and 0.5	Relative critic loss and gradient clipping scale.

Three kinds of clipping must be kept separate. Action clipping enforces the normalized command Box. Observation clipping enforces input bounds. PPO objective clipping affects the training loss. None alone ensures safe behavior, good formation, or small policy changes.

With narrow action noise, even a small absolute action-mean change can be large compared with that noise. Updating a pretrained actor with the old learning rate was therefore especially risky. Reducing the learning rate and adding the KL guard addressed the observed update damage together; our experiment did not isolate their individual contributions.

How neural updates and training diagnostics work

A network contains numerical weights. A loss measures some mismatch: for imitation, the difference from teacher actions; for the critic, disagreement with return targets; for PPO's actor, a reward-based policy objective. A gradient describes which small weight changes locally increase or decrease that loss. An optimizer takes those changes. Adam adapts update scales across weights, but the learning rate still matters.

A minibatch is a small group of collected samples used for one update. An epoch is one pass through a rollout dataset. Ten epochs reuse that rollout up to ten times; they do not collect ten times more robot experience. PPO then needs fresh interaction from its updated policy. Unlike a long-lived replay buffer commonly used in off-policy methods, this PPO workflow updates from its recent rollout buffer.

Gradient clipping limits the combined gradient magnitude before an optimizer step. It is another distinct safeguard: it does not impose command acceleration limits or guarantee a small change in the output action. Our failed warm start had gradient clipping enabled and still changed its action distribution dramatically.

Entropy describes how spread out the action distribution is. Its small bonus encourages some exploration, while sigma gives an easier-to-read per-axis noise scale. Continuous-distribution entropy and its logged negative loss can have unfamiliar signs; neither is an episode success percentage. The relevant question is whether sampled variations allow useful successful experience.

Mean squared error, or MSE, averages squared prediction errors. It is used to assess imitation action fitting and critic return fitting, but those losses predict different quantities and have different units. Explained variance measures how much variability in return the critic captures: one is ideal, zero means little variance explained, and negative values are a warning. A value of 0.25 is not a 25 percent task-success rate.

Approximate KL diagnoses policy distribution change; clip fraction measures how often PPO's objective clipping is active. Neither is the fraction of physical command-limit violations. Reading these together with sigma, actor drift and actual trajectories made the optimizer collapse visible. Current PPO uses no continuous imitation-loss anchor during fine-tuning: the teacher weights provide initialization, while PPO's objective drives later updates.

How imitation supplied a useful starting actor

Pure reinforcement learning starts with little useful behavior and has to discover good actions through reward. For a slow, three-axis escort controller, broad random actions often lose the client before successful stopping is ever experienced. The reward signal then arrives mainly through failures.

Behavior cloning turns teacher decisions into a supervised learning problem. For each observation, the teacher provides a target action. We fit the actor mean to minimize squared differences from those actions. The critic and Gaussian variance were not trained by that supervised actor fit.

Our dataset collection began with teacher-driven trajectories, then added observations reached by the learner itself and requested teacher labels there. This matters because a learner's small errors take it to situations the teacher rarely visits. Giving it examples of recovery from its own visited states reduces that mismatch. This is inspired by dataset aggregation ideas associated with DAgger; we should not claim every detail or guarantee of the original DAgger algorithm.

The recorded diagnostic used 42 full-G1 episodes and 8,096 labelled observation-action rows, with whole episodes held out for supervised checking. It used three rounds of 4,000 actor updates. Adjacent samples from one trajectory were not scattered randomly across training and holdout, which would make the check deceptively easy.

The successful actor archive still contained an untrained critic and unit action standard deviation. Loading it as an ordinary PPO resume would falsely imply trained optimizer and scenario-stream state. We instead implemented explicit **actor initialization**: check hashes and spaces, copy only the actor tensors, create a fresh critic and PPO optimizer, and record the initialization source.

The scientific comparison must retain a frozen imitation-only baseline. Otherwise a successful fine-tuned model could be credited to PPO even when its useful behavior came almost entirely from teacher supervision.

Exploration calibration and critic preparation

Once a useful mean controller exists, we can ask how much random variation it tolerates. We tested the same twelve development scenarios with matched Gaussian noise samples at five standard deviations. The scenario set was balanced across families and kept separate from final-test configurations.

Per-axis normalized sigma	Successes	Interpretation
1.00	0 / 12	Large noise disrupted formation and produced two supervisor failures.
0.37	0 / 12	Formation was relatively close, but stopping holds failed.
0.10	9 / 12	Some gentle-turn cases still failed.
0.05	12 / 12	Largest tested noise level passing the preset gates.
0.02	12 / 12	Also passed, but offered less exploration.

Sigma=0.05 corresponds to command-space standard deviations of roughly 0.03 m/s in forward and lateral commands and 0.045 rad/s in turning before action clipping. It does not mean the actual robot's velocity fluctuates by exactly those amounts; the gait and dynamics determine the response.

Reducing noise alone had previously failed when starting from an unhelpful actor mean. That is not a contradiction: small variations around a bad command remain bad, while small variations around a working controller can preserve useful experience.

Next we froze actor and variance, collected 24 successful training-only episodes, and fitted the critic to discounted returns from 4,467 transitions. Five complete episodes, containing 923 samples, formed a held-out training-data check. This check is separate from the fixed validation suite used to judge the policy.

Critic held-out squared error fell from 134.1 to 10.20, compared with 11.71 for a constant predictor. Explained variance was only 0.13: a modest improvement, not a solved value-estimation problem. We used a separate critic-only Adam optimizer for 6,000 updates; the PPO optimizer remained fresh and the actor stayed exactly unchanged.

The bootstrap is zero at true termination. At a time-limit cutoff, unfinished future value is estimated from the critic. Keeping that distinction prevents a time limit from masquerading as a task-ending failure in the return targets.

Why the earlier failures were useful evidence

The original long run had a confirmed scenario-RNG resume defect: 471 resets in 90k additional transitions produced only 65 unique seeds. Loading PPO queued vector-environment seeds. Although saved worker generators were restored, the first later reset applied those queued seeds and replaced the restored streams.

The fix was an ordering protocol: load the model, clear pending reset seeds and options, restore each worker's saved generator, verify the state, then allow an unseeded reset. No time-based reseeding was used to hide the problem. Tests check expected future scenario sequences, not merely whether weights reload.

Fixing this did not by itself solve learning. Different reward and low-noise-from-scratch runs still struggled. That is why we treated reward alignment, exploration, representation and optimizer behavior as separate hypotheses rather than blaming one cause for every failure.

The decisive warm-start comparison held the successful actor, objective C and sigma=0.05 fixed. The unguarded run used learning rate 0.0003 without a target-KL guard. It went from 60/60 validation successes to 0/60 after 5k PPO transitions. Its first optimizer block had approximate KL 295 and a forward-action RMS shift of 1.21 normalized units on the fixed observation probe.

The failed actor then settled roughly 0.43 m behind the client, rather than 1.5 m. Its terminal speed was near zero. This distinguished a wrong high-level equilibrium from a gait that physically could not stop.

Using learning rate 0.00001 and target_kl=0.01 preserved the actor through 5k and then 10k PPO transitions. The important engineering skill was to isolate mechanisms with controlled tests: representation through imitation, exploration through a frozen-actor sweep, value fit through actor freezing, optimization through a guarded comparison, and resume through exact RNG continuation.

What the current results establish

Common strict validation metric	Imitation actor	Guarded PPO 5k	Guarded PPO 10k
Success	60 / 60	60 / 60	60 / 60
Mean formation error	5.91 cm	5.63 cm	5.77 cm
Mean terminal error	2.01 cm	2.44 cm	2.10 cm
Critical failures	0	0	0

Each policy passed 20 straight, 20 gentle-turn and 20 stop/start cases. These evaluations reuse the same sixty scenarios; two passing rounds do not mean 120 distinct scenarios. There were also 48/48 successful completed stochastic training episodes across the two guarded chunks.

In the resumed chunk, KL stayed around 0.004–0.008 and the guard activated once. There were 157 additional actual Adam steps, bringing the saved total to 351. The actor really changed; it was not accidentally frozen. The complete next scenario sequences matched the saved worker RNG states, with 52 distinct sampled training seeds across the two chunks.

Training total is **10,000 PPO transitions**, plus 4,467 critic-preparation transitions, plus the prior imitation dataset and all evaluation/calibration collection. Do not report 10k as the total amount of supervision or simulator interaction used by the whole method. Ten thousand high-level transitions represent 2,000 aggregate simulated seconds and one million active physics steps, excluding resets and evaluation.

Guarded training throughput was about 66.6 transitions per wall-clock second with two workers. That rate includes optimizer work, rollout logging and automatic training resets, but excludes critic preparation and long validation/plotting phases. GPU speedup cannot be assumed: the small CPU networks consumed little of the measured wall time.

The results establish stable imitation-pretrained PPO fine-tuning for one model seed on this Stage 1 suite. They do not prove substantial improvement over imitation, learning from scratch, real-human safety, or full numerical parity with the live Grove stack. The final-test split remains untouched. Another PPO seed and additional development routes are useful before broad claims.

The selection rule kept the 5k checkpoint as best in the second chunk because success tied and its mean formation error was slightly lower. The 10k final checkpoint remains valid for continuation. Selecting only by reward would miss that distinction.

Why seeds and held out scenarios matter

A random seed is a reproducible starting point for a random-number generator. The generator’s saved state is its current position in the sequence. Reloading the original seed is like reopening a book at page one; restoring the saved state resumes at the page you actually reached.

Separate streams control model initialization/action sampling, worker scenario selection and evaluation. Each worker owns its own physical and controller state as well as its own random stream. Saving neural weights alone does not preserve those streams. Conversely, restoring RNG alone does not restore a robot halfway through a step; our documented resume starts new physical episodes from the next saved scenario samples.

Training scenarios supply gradient updates. Validation scenarios judge checkpoints and guide engineering. Final tests remain untouched until the final comparison. Repeatedly using validation to choose settings can adapt the experiment to that suite, even without directly training on its rewards. This is why an additional development batch and another PPO seed are useful after a 60/60 result.

Different seed numbers do not automatically guarantee meaningfully different routes. The implementation checks resolved geometry, uses disjoint seed namespaces, and reserves a high-speed-and-long-route combination for final testing. Holding out whole episodes during imitation and critic preparation also prevents near-identical adjacent samples from leaking across a split.

A practical checklist for reasoning about the next experiment

1. **Freeze the question.** Decide whether you are testing discovery, preservation, improvement or robustness. Passing a preservation test answers only the preservation question.
2. **Change few variables.** Keep the gait, physical model, observation units and scenarios fixed when comparing PPO settings. Record when a reward or success definition changes.
3. **Separate teacher information from actor information.** The actor and initial critic get the same causal observations. Future geometry can support evaluation without becoming an input.
4. **Check actual behavior.** Look at signed formation errors, commands, achieved velocity, stopping and failure reasons as well as reward and success.

5. **Check state continuity.** Save optimizer and RNG state, validate their restoration, and document that active episodes restart. A loadable ZIP is not a complete resume contract.
6. **Use short evidence-driven runs.** A small test that reveals immediate destructive updates is more useful than repeating a blind 100k run. A larger budget needs a concrete question.
7. **Keep comparisons honest.** Retain imitation-only, label the fine-tuned method, use matched scenarios and reserve final tests for the final scientific comparison.

A checkpoint is the saved network and optimizer state. A manifest is its experiment record: versions, hashes, settings, seeds and source identity. Together with traces and RNG sidecars they make the result reproducible. Checkpointing is therefore part of research correctness, not just backup convenience.

The next curriculum stage adds one pedestrian and creates new tradeoffs. It should preserve some easy Stage 1 sampling to reduce forgetting, while using explicit geometry-based clearance for noncolliding human proxies. Passing Stage 1 is readiness to attempt that next stage, not proof it will work automatically.

Where these ideas come from and where to inspect this project

Schulman and colleagues, Proximal Policy Optimization Algorithms, 2017 introduces PPO's policy-update approach. Our implementation uses Stable-Baselines3 2.7.0 and its actual local source; its optional target-KL stopping behavior is distinct from the clipped objective.

Ross, Gordon and Bagnell, A Reduction of Imitation Learning and Structured Prediction to No-Regret Online Learning, 2011 motivates learning from states visited by the learner. Our teacher-labelled aggregation is described as inspired by that idea, not as a full reproduction of every theoretical condition.

Project definitions live in `research/escort_rl/escort_rl/observations.py`, `reward_variants.py`, `tracking_objective.py`, `imitation_ppo.py`, `ppo_training.py` and `sb3_restart.py`. Configurations are `configs/stage1.yaml`, `tracking_stop_v3.yaml` and `ppo_stage1_imitation_guarded_v1.yaml`.

The measured history is recorded in `PPO_RESUME_REWARD_AUDIT_2026-11-06.md`, `CAPACITY_EXPLORATION_2026-11-06.md`, `IMITATION_PPO_2026-11-06.md`, and `PPO_IMITATION_GUARDED_10K_RESULTS_2026-11-06.md`. These distinguish earlier failed experiments from the current successful preservation result.

Use the separate visualization guide to watch the actual trained controller. A rendered human is a useful illustration of formation, but it does not upgrade our proxy observations into perception or add social response to the client.